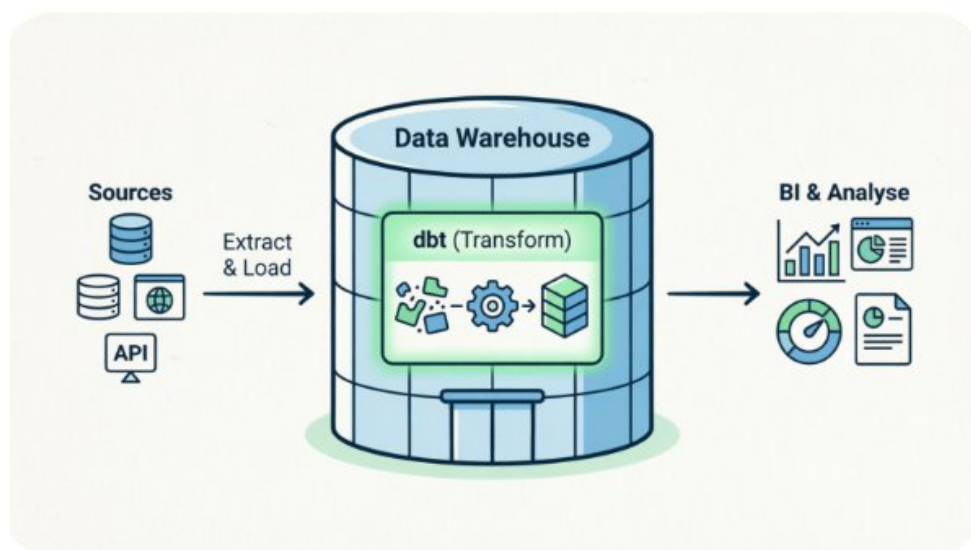


Cours dbt

Data Build Tool

dbt (data build tool) est un outil open-source qui permet de transformer des données directement dans une base de données en écrivant uniquement du SQL. Il gère les dépendances entre les modèles, les tests de qualité, la documentation et la lineage des données. En résumé : dbt prend du SQL, l'exécute dans le bon ordre, et crée des tables ou des vues. C'est tout. Mais cette simplicité cache une puissance considérable.



A retenir : dbt est un outil ELT pur — il transforme les données DANS la base. Il ne les extrait pas et ne les charge pas. C'est la différence fondamentale avec ETL.

Installation et initialisation

Installation

- _ création d'un environnement virtuel
- _ creation d'un requirements.txt (fichier des dépendances)

```
requirements.txt
1 dbt-postgres
2 dbt-core
```

- _ installation des dépendances nécessaires

```
>pip install -r requirements.txt
```

Créer un projet

- _ dbt init (puis répondre aux questions interactivement)
- _ Se placer dans le projet `cd nom_du_projet`
- _ dbt debug pour vérifier la connexion à la base de donnée

Attention : dbt debug doit retourner 'All checks passed' avant de commencer. Si la connexion échoue, vérifier le fichier profiles.yml.

Ce PC > Disque local (C:) > Utilisateurs > [redacted] > .dbt >

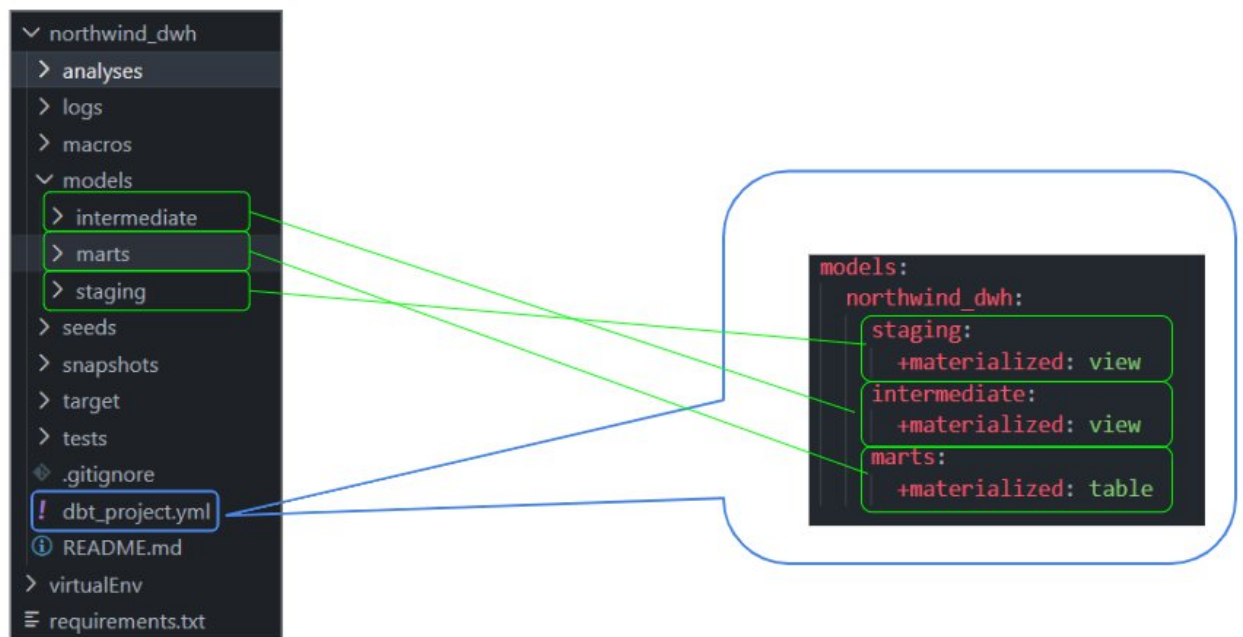
Dans le cadre de notre Briefs dbt il doit ressembler ca :

```
northwind_dwh:
  outputs:
    dev:
      dbname: northwind
      host: localhost
      pass: [redacted]
      port: 5432
      schema: dbt_dev
      threads: 4
      type: postgres
      user: dbt_user
      target: dev
```

dbt créera ce schéma automatiquement

utiliser un user dédié surtout pas postgres (ou super-user)

Structure du projet :



Les modèles

Un modèle dbt est simplement un fichier .sql dans le dossier models/.

dbt exécute ce SQL et crée une table ou une vue dans la base. La magie : dbt gère automatiquement les dépendances entre modèles via les fonctions `ref()` et `source()`.

[source\(\)](#) : référencer une table existante

`source()` fait référence à une table existante dans votre base (vos données OLTP). Elle doit être déclarée dans le fichier `sources.yml`.

```
sources:
  - name: northwind
    schema: public
    tables:
      - name: categories
```

```
select * from {{ source('northwind', 'categories') }}
```

ref() : référencer un autre modèle dbt

ref() fait référencer a un autre modèle dbt. Elle crée une dépendance : dbt exécutera le modèle référencé en PREMIER. C'est le mécanisme qui permet a dbt de gérer l'ordre d'exécution automatiquement.

```
SELECT *  
FROM {{ ref('stg_orders') }} -- execute stg_orders AVANT ce modele  
JOIN {{ ref('stg_customers') }} -- execute stg_customers AVANT ce modele  
ON stg_orders.customer_id = stg_customers.customer_id
```

A retenir : Ne jamais mettre le nom du schema dans ref() ou source(). dbt sait ou trouver les modeles. Ecrire ref('stg_orders') et non ref('dbt_dev.stg_orders').

Les couches de notre projet dbt :

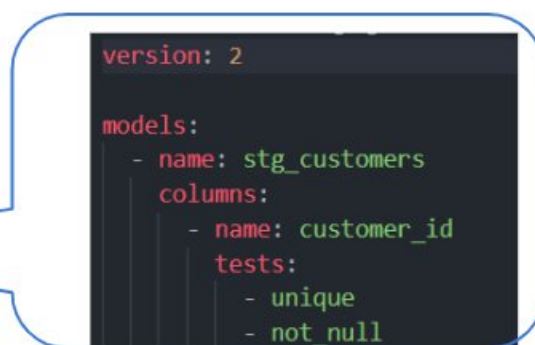
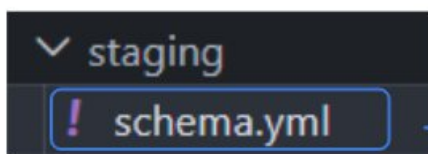
Couche	préfixe	role	<u>materialisation</u>	contenu
Staging	stg_	nettoyer les sources	vue	1 modèle par table source
intermediate	int_	enrichissement	vue	jointure, et préparation pour les marts
marts	dim_ fact_	tables analytiques finales	table	dimensions et tables de fait requêtées par les analystes

Tests de qualité

dbt permet de tester automatiquement la qualité des données. Les tests se définissent dans les fichiers schema.yml et s'exécutent avec dbt test. Un test qui échoue retourne une erreur : dbt ne supprime rien, il vous informe.

Test	Ce qu'il verifie	Exemple d'usage
unique	Pas de doublons dans la colonne	customer_id, order_id, product_id
not_null	Aucune valeur NULL dans la colonne	Toutes les colonnes cles
accepted_values	Valeurs limitees a une liste	statut IN ('livre','annule'...)
relationships	Integrite referentielle (FK)	fact.customer_id existe dans dim_customers

Là où s'écrivent les tests



Attention : Chaque test doit être sur une ligne séparée — pas en liste inline. La syntaxe correcte est une ligne par tiret, pas [unique, not_null] sur la même ligne.

Commandes essentielles

Commande	Ce qu'elle fait	Quand l'utiliser
dbt debug	Vérifie la connexion et la configuration	Toujours en premier
dbt run	Exécute tous les modèles	Pour tout reconstruire
dbt run --select stg_orders	Exécute un seul modèle	Pour tester un modèle spécifique
dbt run --select staging.*	Exécute tous les modèles d'un dossier	Pour tester une couche
dbt test	Exécute tous les tests	Après chaque dbt run
dbt test --select stg_orders	Teste un seul modèle	Pour déboguer un test
dbt build	dbt run + dbt test dans le bon ordre	En production
dbt docs generate	Génère la documentation	Avant dbt docs serve
dbt docs serve	Lance le serveur de documentation	Pour voir le lineage
dbt clean	Supprime le cache	En cas de problème